二、课程内容

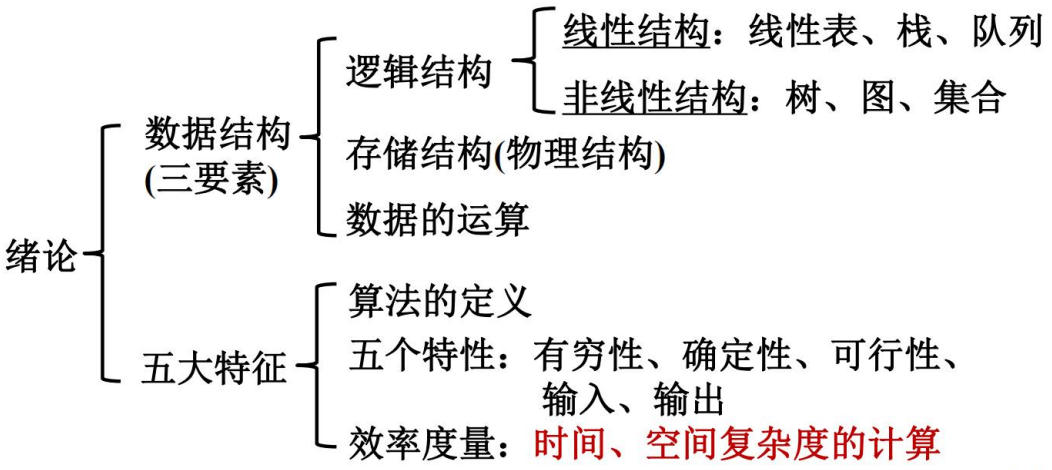
章节	主要内容	授课时数	实验时数
第一章 绪论	①数据结构基本概念和研究方法 ②算法与算法分析	4	0
第二章 线性表	①线性表基本概念 ②线性表存储结构及操作 ③栈与队列 ④字符串 ⑤数组	8	4
第三章 树	①树及二叉树基本概念 ②树及二叉树存储结构及操作 ③线索二叉树 ④树的应用	8	4
第四章 图	①图的基本概念 ②图的存储结构及操作 ③图的连通性（最小生成树） ④有向无环图（拓扑排序、关键路径） ⑤最短路径	8	4
第五章 查找	①静态查找 ②动态查找 ③哈希表	6	0
第六章 排序	①插入排序 ②选择排序 ③快速排序 ④归并排序 ⑤基数排序	4	4
第七章 文件与外排	①顺序文件 ②索引文件 ③直接存取文件 ④多关键字文件	2	0

考纲内容

（一）数据结构的基本概念及相关术语

（二）数据结构的三要素：逻辑结构、物理结构和数据运算

（三）算法时间复杂度和空间复杂度的分析与计算



1.1 数据结构研究对象

1、数据结构研究方法

逻辑结构： 对操作对象的一种数学描述，或从操作对象中抽象出的数学模型，用以描述数据元素之间的逻辑关系，与存储方式无关。

物理结构： 数据结构在计算机中的表示或映象，逻辑结构的存储方式。
又称存储结构，分为顺序映象和非顺序映象；
或称顺序存储结构和链式存储结构；

计算机解题过程：

问题 → 数学模型 → 算法 → 程序 → 测试 → 计算
↳分析数据并确定存储结构 ↑ 输入到计算机中

1.3 抽象数据型Abstract Data Types（ADT）

抽象： 从众多事物中舍弃个别的、非本质的属性，抽出共同的、本质性的特征。
是形成概念的重要手段，其目的是使复杂度降低。

＊软件系统由数据结构、操作过程和控制机能三者组成，软件设计是对三者的抽象过程，即数据抽象、过程抽象和控制抽象。

【定义】抽象数据型是一个数学模型和在该模型上所定义的操作的集合。

ADT特点： ①降低了软件设计的复杂性；
②提高了程序的可读性和可维护性；
③程序的正确性容易保证。

（1）可以用（ ）定义一个完整的数据结构。

A. 数据元素 B. 数据对象 C. 数据关系 D. 抽象数据类型

（2）以下数据结构中，（ ）是非线性数据结构。

A. 树 B. 字符串 C. 队列 D. 栈

（3）以下数据逻辑结构的是（ ）。

A. 顺序表 B. 哈希表 C. 有序表 D. 单链表

（4）链式存储设计时，节点内的存储单元地址（ ）。

A. 一定连续 B. 一定不连续 C. 不一定连续 D. 部分连续，部分不连续

（5）以下关于数据结构的说法，正确的是（ ）。

A. 数据的逻辑结构独立于其存储结构

B. 数据的存储结构独立于其逻辑结构

C. 数据的逻辑结构唯一决定了其存储结构

D. 数据结构仅由其逻辑结构和存储结构决定

同一个逻辑结构，可以对应很多不同的存储方法。逻辑结构本身跟你打算怎么存是**没有关系**的，所以称为“独立于”存储结构。

时间复杂度与空间复杂度的分析

体现了对函数总体渐进增长趋势的关注和刻画

时间

$$T(n) = O(f(n))$$

空间

$$S(n)=O(g(n))$$

表示 T(n)与 f(n)同一级别（感觉可以说成 等价无穷大）

循环主体中的变量参与循环条件的判断

此类题目应该找出主体语句中与T(n)成正比的循环变量，将之代入条件中计算。

设共执行t次

例：以下算法的时间复杂度是（）

```
void func(){int i=0; while(i*i*i<=n) i++;}
```

A. $O(\log_2 n)$　　B. $O(n^{1/2})$　　C. $O(n^{1/3})$　　D. $O(n\log_2 n)$

n 很大时，只用考虑循环中语句的时间复杂度即“i++”

自己来模拟程序的执行，设某一时刻“i++”执行了 t 次

则 i 变成了 0 + t = t 此时判断循环是否结束即判断 t*t*t>n 是否成立

临界情况 t*t*t=n 即 t = n~(1/3)

时间复杂度可以理解成消耗时间的多少，直接影响因素是语句执行次数，也就是 t

（1）某算法的时间复杂度为$O(n^2)$，表明该算法（ ）

A. 问题规模是n^2　　B. 执行时间是n^2

C. 执行时间和n^2成正比　D. 问题规模与n^2成正比

n 就是问题规模

考纲内容

（一）线性表的定义和基本操作（ADT）

（二）线性表的实现

(1) 顺序存储 ---顺序表

(2) 链式存储 ---单链表、双链表、循环链表、静态链表

(3) 线性表的应用

（三）栈和队列

(1) 基本概念

(2) 顺序存储、链式存储

(3) 栈和队列的应用

（四）数组

一维数组、特殊矩阵的压缩存储、稀疏矩阵

2.1 抽象数据型线性表

【定义】线性表是由n(n≥0)个相同类型的元素组成的有序集合。

记为：表头元素 表尾元素

(a_1, a_2, a_3, ……a_{i-1}, a_i, a_{i+1}, …… a_n)

a_1 直接后继 a_{i+1}；a_n 直接前驱 a_{i-1}

其中：① n为线性表中元素个数，称为线性表的长度；

② a_i为线性表中的元素，类型定义为ElementType；

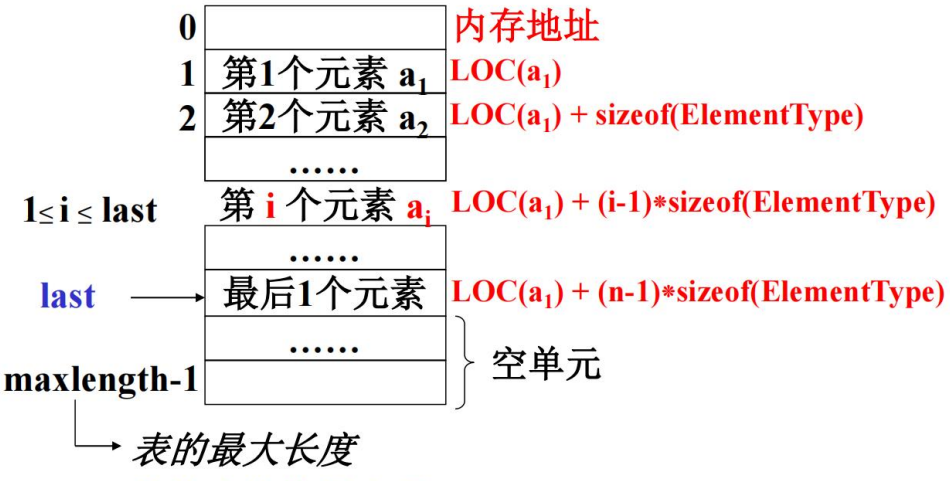
③ a_1为表中第1个元素，无前驱元素；a_n为表中最后一个元素，无后继元素；对于…a_{i-1},a_i,a_{i+1}…(1<i<n)，称a_{i-1}为a_i的直接前驱，a_{i+1}为a_i的直接后继；

④ 线性表是有限的，也是有序的。

⑤ 元素具有逻辑上的顺序性，各元素排序有先后次序。

2.2.2 顺序表--线性表的顺序表示

线性表的顺序存储又称顺序表。使用一组地址连续的存储单元依次存储线性表中的数据元素，从而使得逻辑上相邻的数据元素在物理位置上也相邻。



线性表的数组实现

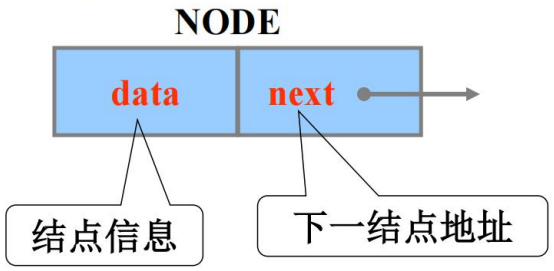
线性表的顺序存储与实现

顺序表的特点：

1. 通过元素的存储顺序反应线性表中数据元素之间的逻辑关系
2. 可随机存取顺序表的元素
3. 顺序表的插入和删除操作要通过移动元素实现

2.2.3 单链表--线性表的指针实现

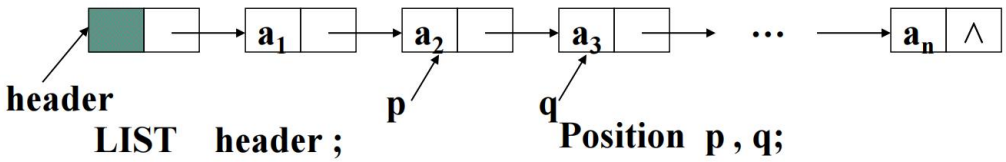
结点形式



结点类型

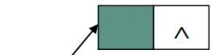
```
struct  NODE {
    ElementType  data ;
    struct NODE  *next ;
} ;

typedef  NODE  *LIST;
typedef  NODE  *Position;
```



header　　p　　q

LIST　header ;　　Position　p , q;

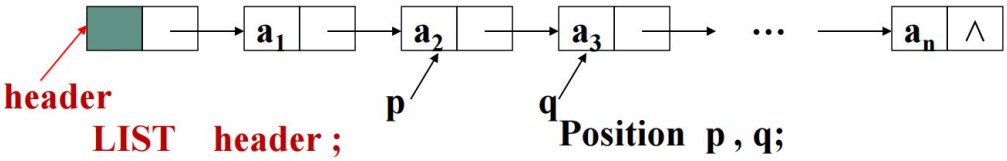


header

记法：　a_2: (*p).data ;　⇒　a_2: p→data ;

q: (*p).next ;　　q: p→next ;

2.2.3 单链表--线性表的指针实现



头结点与头指针的区别：

不管有没有头结点，头指针始终指向链表的第一个结点；

而头结点是带有头结点的链表中第一个结点，通常不存储任何信息。

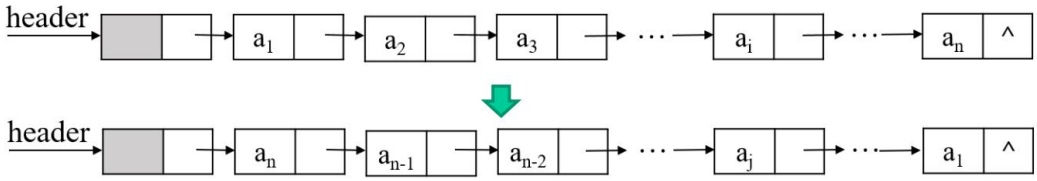
引入头结点的好处：

① 头结点的指针域存放首结点的位置，链表第一个位置和其他位置上的操作一致，无需特殊处理。

② 无论链表是否为空，其头指针都指向头结点的非空指针（空表中头结点的指针域为空），空表与非空表处理统一。

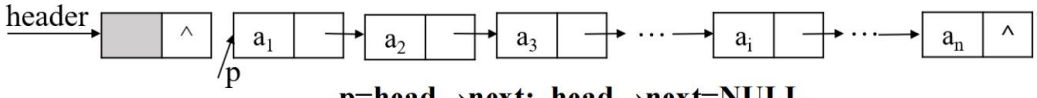
头插法实现链表反向

【例2-4】线性表的反向，即线性表的最后一个元素变成第一个元素，而第一个元素变成最后一个元素。



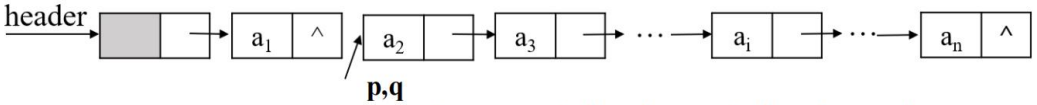
分析：

第1步：



p=head→next; head→next=NULL

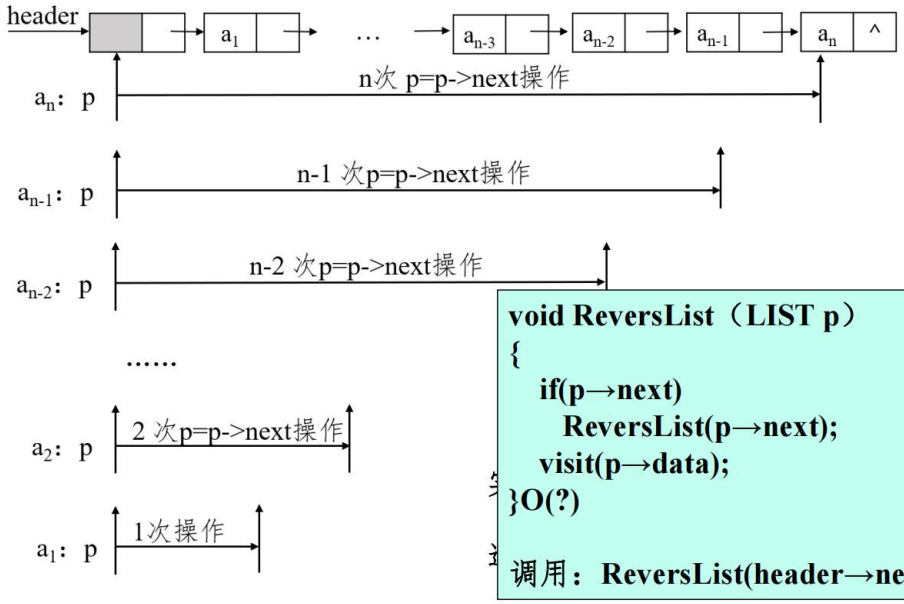
第2步：



q=p→next; p→next=head→next; head→next=p; p=q;

递归

【例2-5】按逆序遍历链表




```
void ReversList（LIST p）
{
   if(p→next)
     ReversList(p→next);
   visit(p→data);
}O(?)

调用：ReversList(header→next);
```

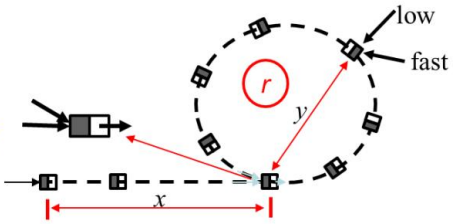

思考题：

1、单向链表环的问题

如何判断一个单向链表是否有环？找到环的入口结点。

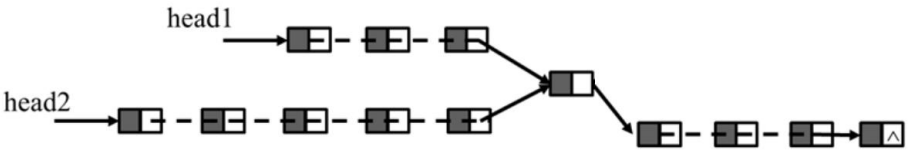
2、线性链表，求倒数第k个数

3、线性链表，求中间位置的元素



4、单向链表交叉问题

如何判断两个单向链表是否交叉？找到交叉结点。



1，快慢指针，一个 low 速度为 1，一个 fast 速度为 2，重合则有环。找环的入口有数学推导 https://zhuanlan.zhihu.com/p/454449703，做法为当 low 和 fast 相遇时，再用一个指针 p 指向头结点，p 和 low 都以速度 1 遍历，p 和 low 相遇点即为环的入口

2，快慢指针，快指针先走 k 步，然后快慢指针同速遍历，快指针到表尾时慢指针所指即为所求

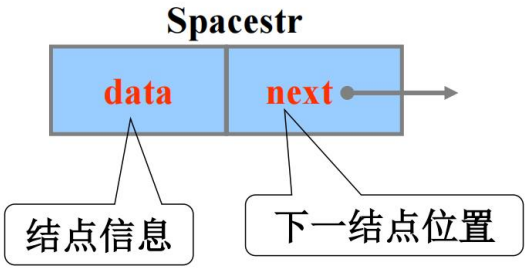
3，快慢指针，一个速度为 1，一个速度为 2

4，先把两个链表都各遍历一遍求出长度 m 和 n，不妨设 m<=n。再用指针 p1, p2 指向表头，p2 先走 n-m 步，然后 p1 和 p2 同时继续遍历，若遍历过程中 p1 与 p2 重合则交叉。

2.2.4 静态链表--线性表的游标实现

借助数组来描述线性表的链式存储结构，节点包含data和next。

结点形式



结点类型

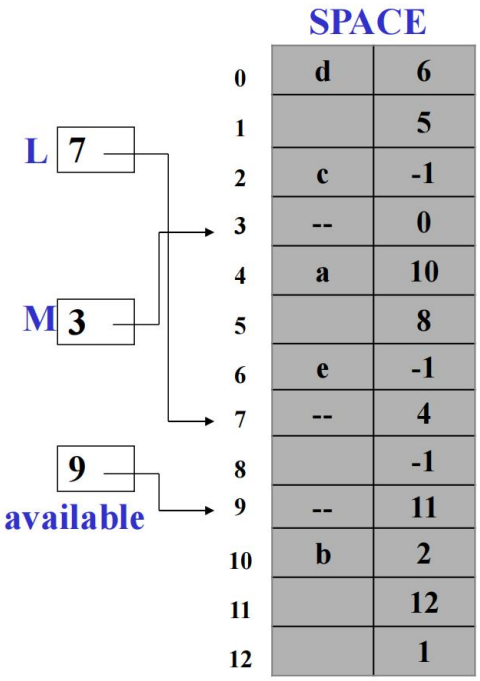
```
struct  Spacestr {
    ElementType  data ;
    int    next ;
} ;
```

线性表：

```
Spacestr  SPACE[ maxsize ] ;
typedef  int  Position;
```

元素：SPACE[i].data → “型”为ElementType(基本、复合)

下一元素位置：SPACE[i].next →“型”为 int



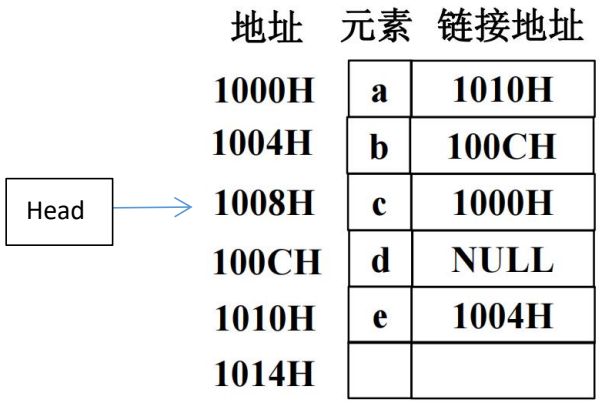
线性表有哪些呢？
线性表1：（a, b, c）　L = 7
线性表2：（d, e）　M = 3
空闲表：available = 9

```
x = New Spacestr ;//自己定义
    x = SPACE[available].next ;
    SPACE[available].next =
        SPACE[ x ].next ;

Delete  x ;
    SPACE [ x ].next =
        SPACE[available].next ;
    SPACE[available].next = x ;
```


例题

已知表头元素是c的单链表在内存中的存储状态如下表所示。



地址	元素	链接地址
1000H	a	1010H
1004H	b	100CH
1008H	c	1000H
100CH	d	NULL
1010H	e	1004H
1014H		

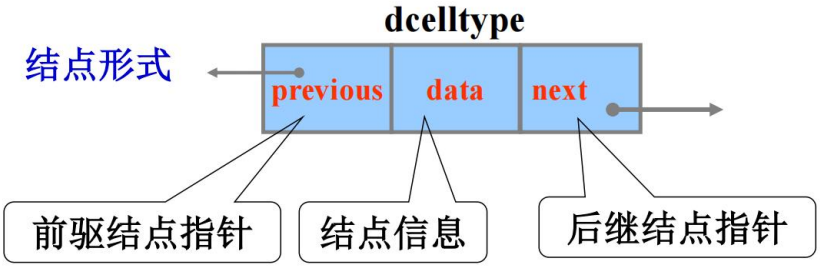
现将f存放于1014H处并插入到单链表中，若f在逻辑上位于a与e之间，则a，e，f的“链接地址”依次是（）

A. 1010H, 1014H, 1004H　　B. 1010H, 1004H, 1014H
C. 1014H, 1010H, 1004H　　D. 1014H, 1004H, 1010H

原：　c->a->e->b->d
插入后：c->a->f->e->b->d
注意问的顺序

2.2.5　双向链表

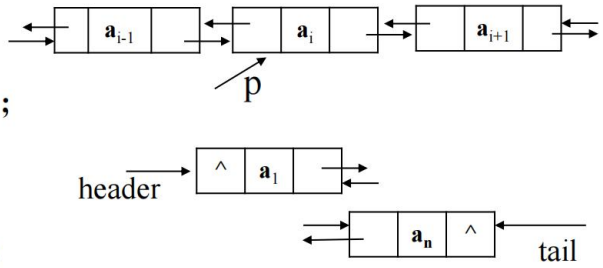
结点形式



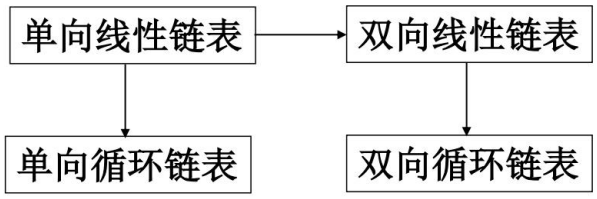
结点类型

```
struct DnodeType {
   ElementType data ;
   DnodeType *next, *previous ;
} ;

typedef DnodeType *DLIST ;
typedef DnodeType *Position ;
```

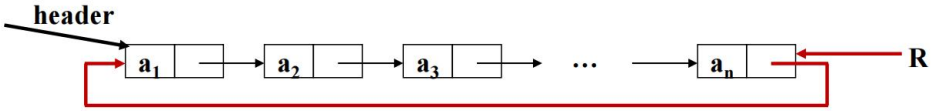


2.2.6　环形链表



对线性链表的改进，解决“单向操作”的问题；改进后的链表，能够从任意位置元素开始访问表中的每一个元素。

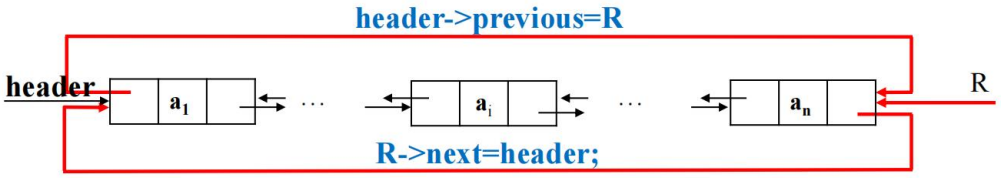
单向环形链表：



其结构与单向线性链表相同，用表尾指针标识链表，从而省去了表头结点。

表头：R→next　亦即表中的一个元素a_1

双向环形链表

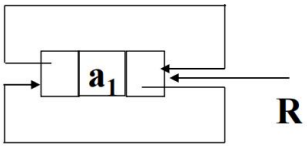


双向环形链表的结构与双向连表结构相同，只是将表头元素的空previous域指向表尾，同时将表尾的空next域指向表头结点，从而形成向前和向后的两个环形链表，对链表的操作变得更加灵活。

空双向环形链表：

R=Null;

单一结点双向环形链表：



R->next=R;
R->previous=R;

例题 已知一个带有表头节点的双向链表L，结点结构为

prev	data	next

其中，prev和next分别是指向其直接前驱和直接后继结点的指针。现要删除指针p所指向的结点，正确的语句是（ ）

A. p->next->prev = p->prev; p->prev->next = p->prev; free(p);

B. p->next->prev = p->next; p->prev->next = p->next; free(p);

C. p->next->prev = p->next; p->prev->next = p->prev; free(p);

D. p->next->prev = p->prev; p->prev->next = p-> next; free(p);

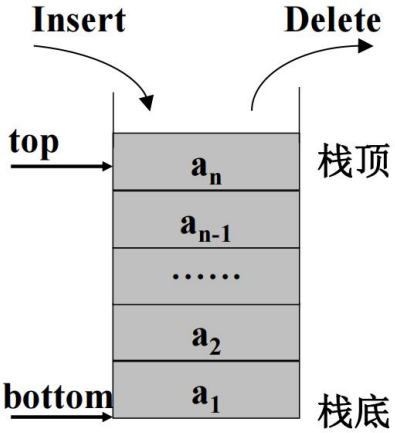
2.3 栈（Stack）

栈是线性表的一种特殊形式，是一种限定性数据结构，也就是在对线性表的操作加以限制后，形成的一种新的数据结构。

【定义】栈是限定只在表尾进行插入和删除操作的线性表。

栈又称为后进先出(Last In First Out)的线性表。 简称LIFO结构。

栈举例.....



栈示意图

2.3.1 栈的实现

1、顺序存储

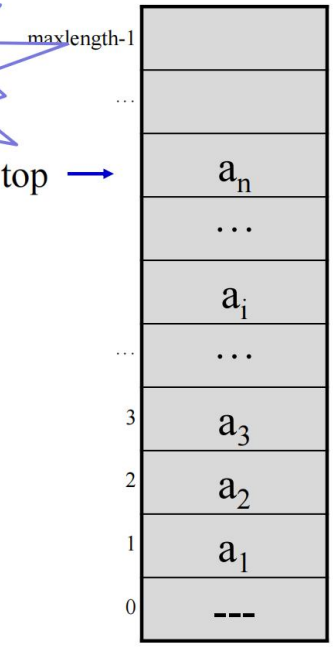
注意约定栈顶指针位置

结构类型：

```
typedef struct {
   ElementType data[maxlength];
   int top;
} STACK;

STACK S;
```

栈的容量：maxlength – 1 ;
栈空：S.top = 0 ;
栈满：S.top = maxlength – 1 ;
栈顶元素：S.data[S.top] ;



顺序栈示意图

这是从下标 1 开始存的

【例】用 S 表示进栈操作，用 X 表示出栈操作，若元素的进栈顺序是 1234，为了得到1342的出栈顺序，相应的 S 和 X 的操作序列为（ ）

A. SXSXSSXX　B. SSSXXSXX
C. SXSSXXSX　D. SXSSXSXX

【例】若元素a, b, c, d, e, f, g依次进栈，允许进栈、退栈操作交替进行，但不允许连续3次进行退栈操作，不可能得到的出栈序列是（ ）

A. dcebfa　B. cbdaef　C. bcaefd　D. afedcb

波兰逻辑学家J.Lukasiewicz于1929年提出的一种表达式。

3、表达式求值

表达式：前缀表达式（波兰式）
中缀表达式
后缀表达式（逆波兰式）

如：

$(a+b)*(a-b)$ 的三种表达式：

*+ab−ab
(a+b)*(a−b)
ab+ab−*

高级语言中，采用类似自然语言的中缀表达式，但计算机对中缀表达式的处理是很困难的，而对后缀或前缀表达式则显得非常简单。

后缀表达式的特点：

① 在后缀表达式中，变量（操作数）出现的顺序与中缀表达式顺序相同。

② 后缀表达式中不需要括弧定义计算顺序，而由运算（操作符）的位置来确定运算顺序。

I. 将中缀表达式转换成后缀表达式

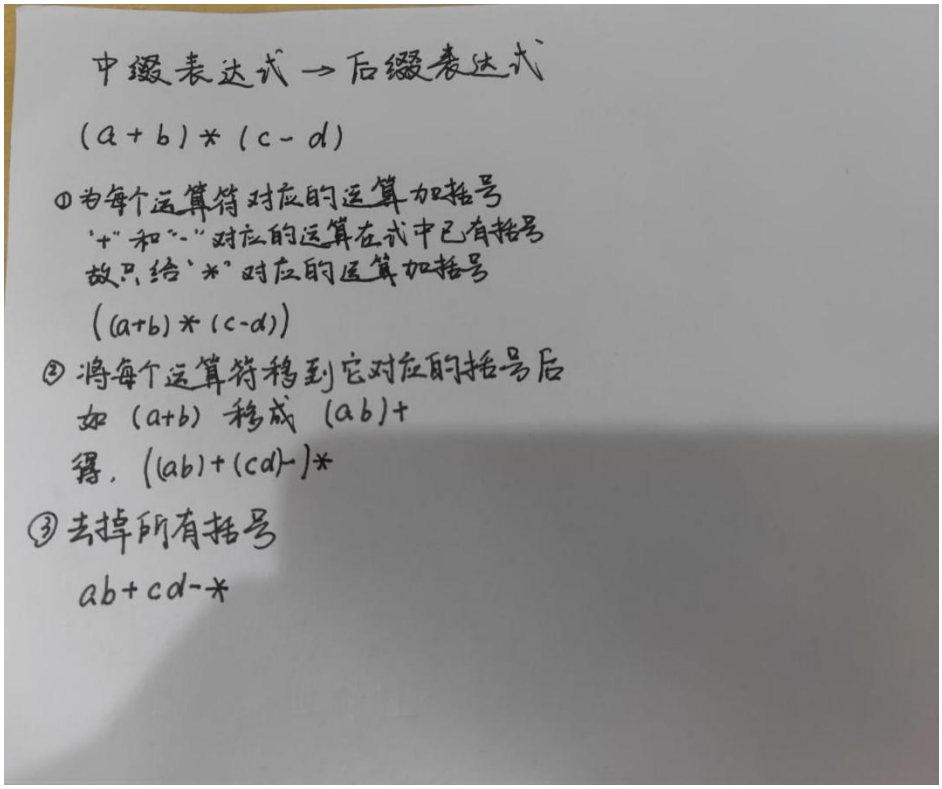
对中缀表达式从左至右依次扫描每一个字符，由于操作数的顺序保持不变，当遇到操作数时直接输出；为调整运算顺序，设立一个栈用以保存操作符，扫描到操作符时，将操作符压入栈中，进栈的原则是保持栈顶操作符的优先级要高于栈中其他操作符的优先级，否则，栈顶操作符依次退栈并输出，直到表达式结束。遇到“(”进栈，当遇到“)”时，退栈输出直到“(”为止，括号内部依然按照前述操作。

II. 由后缀表达式计算表达式的值

对后缀表达式从左至右依次扫描，与I相反，遇到操作数时，将操作数进栈保留；当遇到操作符时，从栈中退出两个操作数并作相应运算，将计算结果进栈保留；直到表达式结束，栈中唯一元素即为表达式的值。

中缀表达式转化成后缀表达式：从左到右扫描，遇到数直接写，遇到操作符进栈。操作符必须优先级高于栈中同区域的所有元素才能直接进栈，否则先把该区域输出完再进栈。“（”用来划分区域，“）”的出现则将该区域全部出栈。

简易方法：为每个运算符对应的运算添括号，再将所有运算符移直自己的括号后，去掉所有括号即可



例题 已知操作符包括+、-、*、/、（和）。将中缀表达式a+b-a*((c+d)/e-f)+g 转换成等价的后缀表达式ab+acd+e/f-*-g+时，用栈来存放暂时还不能确定运算次序的操作符。若栈初始为空，则转换过程中同时保存在栈中的操作符的最大个数是（ ）

A. 5 B. 7 C. 8 D. 11

例题 假设栈初始为空，将中缀表达式a/b+(c*d-e*f)/g 转换成等价的后缀表达式时，当扫表到f时，栈中元素依次是（ ）

A. +(*- B. +(-* C. /+(*-* D. /+-*

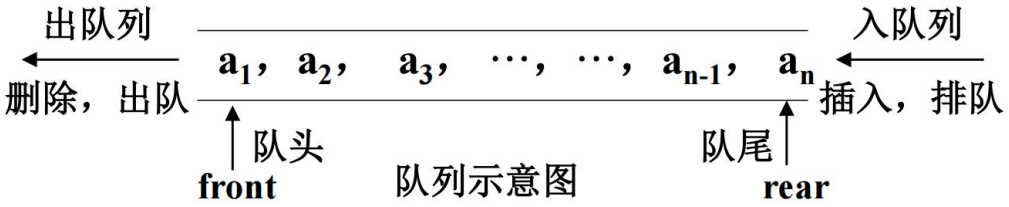
a+b-a*((c+d)/e-f)+g

从左到右遍历		结果	栈
a	数直接输出	a	空
+	运算符进栈	a	+
b	输出	ab	+
-	进栈X 优先级不比"+"高	ab+	-
	退栈完再进栈		
a	输出	ab+a	-
*	进栈	ab+a	-*
(	进栈	ab+a	-*(
(	进栈	ab+a	-*((
c	输出	ab+ac	-*((
+	进栈	ab+ac	-*((+
d	输出	ab+acd	-*((+
)	退栈	ab+acd+	-*(
/	进栈	ab+acd+	-*(/
e	输出	ab+acd+e	-*(/
-	优先级不高 退栈再进栈	ab+acd+e/	-*(-
f	输出	ab+acd+e/f	-*(-
)	退栈	ab+acd+e/f-	-*
+	优先级不高 退完栈再进栈	ab+acd+e/f-*-	+
g	输出	ab+acd+e/f-*-g	+
结束	退栈	ab+acd+e/f-*-g+	空.

2.4 队列（Queue）

队列是对线性表的插入和删除操作加以限定的另一种限定型数据结构。

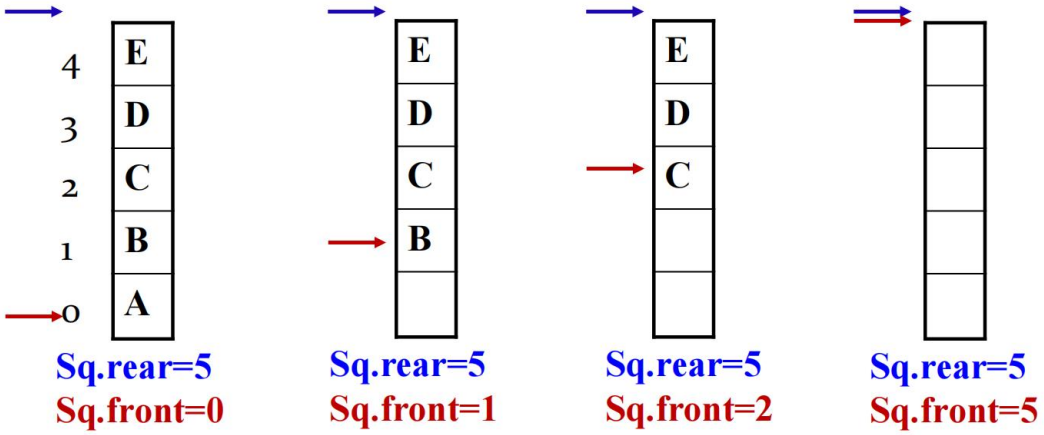
【定义】将线性表的插入和删除操作分别限制在表的两端进行，和栈相反，队列是一种先进先出（First In First Out，简称 FIFO 结构）的线性表。



队列示意图

ADT操作： MakeNull(Q)、Front(Q)、EnQueue(x, Q)、DeQueue(Q)、Empty(Q)

- 顺序队列讨论：



讨论结论：在采用一般顺序队列时出现假溢出现象？

当元素被插入到数组中下标最大的位置上之后，队列的空间就用尽了，但此时数组的低端还有空闲空间，这种现象叫做假溢出。

解决假溢出的方法有多种；一是通过不断移动元素位置，每当有元素出队列时，后面的元素前移一个位置，使队头元素始终占据队列的第一个位置。

第二种方法是采用循环队列。

插入元素：

Q.rear = (Q.rear + 1） % maxlength

删除元素：

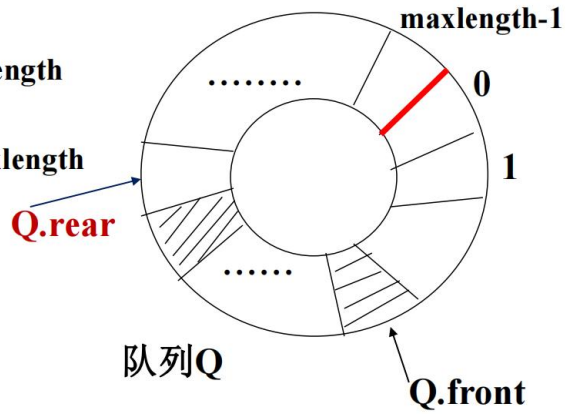
Q.front = (Q.front + 1） % maxlength

队空：

Q.rear == Q.front

队满：

Q.rear == Q.front



问题：如何解决循环队列中队空与队满状态相同？

方法一：另设一个标志位用以区别队空与队满两种状态(count)；

方法二：约定队列头指针在队列尾指针的下一位置上(空位)；

队空：Q.rear == Q.front

队满：(Q.rear+1) mod maxlength == Q.front

结论：两种方法的代价是相同的。

ADT操作：

```
int Addone( int i )
{ return ( ( i + 1)% maxlength ) ; }
```

①
```
Void MakeNull ( QUEUE &Q)
{ Q.front = 0 ;
  Q.rear = maxlength – 1; }
```

②
```
boolean Empty( Q )
  QUEUE Q ;
{ if (Q.rear == Q.front )
      return TRUE ;
  else      return FALSE ;
}
```

③
```
ElementType Front( QUEUE Q )
{if ( Empty( Q ) )
     return Null ;
  else
     return (Q.data[ Q. front ] );
} ;
```

例：已知循环队列顺序存储A[0…n-1]，队列非空时front和rear分别指向队头和队尾元素。若初始为空队列，且要求第一个进入队列的元素存在A[0]的位置，那么front和rear初始值（ ）

（A） 0,0 （B） 0，n-1 （C） n-1,0 （D） n-1,n-1

3、模式匹配(朴素模式匹配算法)

模式匹配(字符串匹配是计算机的基本任务之一)

给定S=“S_0 S_1… S_{n-1}”(主串)和T=“T_0 T_1…T_{m-1}”(模式)，在S 中寻找T 的过程称为模式匹配。如果匹配成功，返回T 在S 中的位置；如果匹配失败，返回-1。

假设串采用顺序存储结构。

朴素模式匹配算法(Brute-Force算法) ：枚举法(回溯)

从主串S 的第一个字符开始和模式T 的第一个字符进行比较，若相等，则继续比较两者的后续字符；否则，从主串S 的第二个字符开始和模式T 的第一个字符进行比较。

重复上述过程，直到T 中的字符全部比较完毕，说明本趟匹配成功；或S 中字符全部比较完，则说明匹配失败。

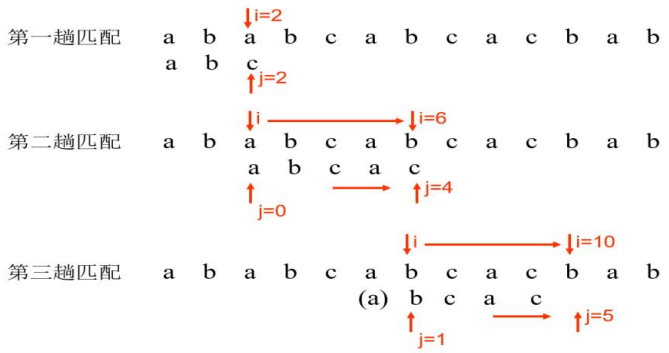
设主串S=“ababcabcacbab”，模式串T=“abcac”

第1趟匹配	主串	ababcabcacbab	i=2
	模式串	abc	j=2 匹配失败
第2趟匹配	主串	ababcabcacbab	i=1
	模式串	abc	j=0 匹配失败
第3趟匹配	主串	ababcabcacbab	i=6
	模式串	abcac	j=4 匹配失败
第4趟匹配	主串	ababcabcacbab	i=3
	模式串	abc	j=0 匹配失败
第5趟匹配	主串	ababcabcacbab	i=4
	模式串	abc	j=0 匹配失败
第6趟匹配	主串	ababcabcacbab	i=9 //返回i-lenT+1
	模式串	abcac	j=4 匹配成功

特点:主串指针需回溯（i-j+1），模式串指针需复位（j=0）。

2.5.2 KMP算法----改进的模式匹配算法

- 为什么BF算法时间性能低？
  在每趟匹配不成功时存在大量回溯，没有利用已经部分匹配的结果；
- 如何在匹配不成功时主串不回溯？
  主串不回溯，模式就需要向右滑动一段距离；
- 如何确定模式的滑动距离？
  利用已经得到的“部分匹配”的结果；
  将模式向右“滑动”尽可能远的一段距离(next[j])后，继续进行比较；
- 字符串的前缀和后缀：（主串ababcabcacbab,模式abcac)



next函数（换一种定义）：

(1)next[0]= -1，任何串的第一个字符的模式值规定为-1

(2)next[j] = -1，模式串t中下标为j的字符，如果与首字符相同，且j的前面的1…k 个字符与开头的1..k 个字符不等(或者相等但t[k]==t[j])，1≤k<j

如：t="abCabCad" 则 next[6]=-1，因t[3]=t[6]

(3)next[j] = k，模式串t中下标为j的字符，如果j的前面k个字符与开头的k个字符相等，且$t_j \neq t_k$，1≤k<j，即

"$t_0t_1t_2\ldots,\ldots,t_{k-1}$" = "$t_{j-k}t_{j-k+1}t_{j-k+2}\ldots t_{j-1}$"

且 $t_j \neq t_k$，1≤k<j

(4)next[j]=0，除(1)、(2)、(3)的其他情况。

从左到右遍历，遍历到哪一位表示这一位失配了。故在他前面找首尾最大匹配长度 k，且必须保证两个对应上的片段后紧跟着的一位不同，则 next[j] = k。否则为零。首位为-1，与首位相同的元素的 next 值若为 0 则改成-1

【例2-18】求T="ababcaabc" 的模式函数的值

next[0]= -1 根据 (1)
next[1]= 0 根据 (4)
next[2]= -1 根据 (2)
next[3]= 0 根据 (3) 虽T[0]=T[2] 但T[1]=T[3] 被划入 (4)
next[4]= 2 根据 (3) T[0]T[1]=T[2]T[3] 且T[2] !=T[4]
next[5]= -1 根据 (2)
next[6]= 1 根据 (3) T[0]=T[5] 且T[1]!=T[6]
next[7]= 0 根据 (3) 虽T[0]=T[6] 但T[1]=T[7] 被划入 (4)
next[8]= 2 根据 (3) T[0]T[1]=T[6]T[7] 且T[2] !=T[8]

即：

下标	0	1	2	3	4	5	6	7	8
T	a	b	a	b	c	a	a	b	c
next	-1	0	-1	0	2	-1	1	0	2

next[3]=0，而不是=1；
next[6]=1，而不是= -1；
next[8]=2，而不是= 0；

2、数组的压缩存储

(1) 特殊矩阵

若n 阶矩阵 A 中的元素满足下述性质

$$a_{ij} = a_{ji} \qquad 1 \le i, j \le n$$

则称 n 阶对称阵。

对于对称矩阵, 为实现节约存储空间, 我们可以为每一对对称元素分配一个存储空间, 这样, 原来需要的 n^2 个元素压缩存储到 n(n+1)/2 个元素空间。

对称关系：

设Sa[1 . . n(n+1)/2] 做为 n 阶对称阵 A 的存储结构, 则 Sa[k] 和 a_{ij} 的一一对应关系为:

$$k = \begin{cases} i(i-1)/2 + j & \text{当 } i \ge j \\ j(j-1)/2 + i & \text{当 } i < j \end{cases}$$

对称矩阵

$$A = \begin{pmatrix} 3 & 6 & 4 & 7 & 8 \\ 6 & 2 & 8 & 4 & 2 \\ 4 & 8 & 1 & 6 & 9 \\ 7 & 4 & 6 & 0 & 5 \\ 8 & 2 & 9 & 5 & 7 \end{pmatrix}$$

对称矩阵特点：$a_{ij}=a_{ji}$

https://blog.csdn.net/tongkongyu

(2) 对角（带状）矩阵

所有非零元素都集中在以主对角线为中心的带状区域内。

$$* \begin{pmatrix} a_{11} & a_{12} & a_{13} & 0 & 0 & 0 \\ a_{21} & a_{22} & a_{23} & a_{24} & 0 & 0 \\ a_{31} & a_{32} & a_{33} & a_{34} & a_{35} & 0 \\ 0 & a_{42} & a_{43} & a_{44} & a_{45} & a_{46} \\ 0 & 0 & a_{53} & a_{54} & a_{55} & a_{56} \\ 0 & 0 & 0 & a_{64} & a_{65} & a_{66} \end{pmatrix} *$$

（S = 2）

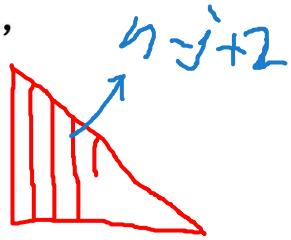
如图所示，S = 2，称S为带宽，为实现压缩存储，我们只存储带区内的非零元素。不难总结出：

LOC[4,3]怎么计算？

LOC[i，j] = LOC[1，1]
+ [(2S+1)*(i-1)+(j-i)]*L

L = 1; //每个数据元素的字节数

例题：若将n阶下三角矩阵A按列优先顺序压缩存放在一维数组B[1…n(n+1)/2]中，则存放到B[k]中的非零元素a_{ij}(1≤i，j≤n)的下标i、j与k的对应关系是（ ）



A. (j-1)(2n-j+1)/2+i-j　　B. (j-1)(2n-j+2)/2+i-j+1
C. (j-1)(2n-j+2)/2+i-j　　D. (j-1)(2n-j+1)/2+i-j-1

例题：有一个 100 阶的三对角矩阵 M，其元素 $m_{i,j}$（1≤i≤100,1≤j≤100）按行优先依次压缩存入下标从 0 开始的一维数组 N 中。元素 $m_{30,30}$在 N 中的下标是（ ）

A. 86　　B. 87　　C. 88　　D. 89

(3) 稀疏矩阵

稀疏矩阵中，零元素的个数远远多于非零元素的个数。为实现压缩存储，我们仍考虑只存储非零元素。行号递增，列号不减。

$$M=\begin{pmatrix}0&12&9&0&0&0&0\\0&0&0&0&0&0&0\\-3&0&0&0&0&14&0\\0&0&24&0&0&0&0\\0&18&0&0&0&0&0\\15&0&0&-7&0&0&0\end{pmatrix}\qquad T=M^T=\begin{pmatrix}0&0&-3&0&0&15\\12&0&0&0&18&0\\9&0&0&24&0&0\\0&0&0&0&0&-7\\0&0&0&0&0&0\\0&0&14&0&0&0\end{pmatrix}$$

a.data

	i	j	v
1	1	2	12
2	1	3	9
3	3	1	-3
4	3	6	14
5	4	3	24
6	5	2	18
7	6	1	15
8	6	4	-7

⇒

b.data

i	j	v
1	3	-3
1	6	15
2	1	12
2	5	18
3	1	9
3	4	24
4	6	-7
6	3	14

```
#define MAXSIZE 100
typedef struct {
    int i, j;
    ElementType e;
} Triple;
typedef struct {
    Triple data[MAXSIZE+1];
    int mu, nu, tu;
} TSMatrix;
```

2.7 广义表（Lists）

广义表是线性表的一种推广结构，线性表要求是相同的数据元素，而广义表中的元素可以取不同类型，可以是最基本的不可再分割的“原子”，也可以是广义表本身。广义表是由零个原子，或若干个原子或若干个广义表组成的有穷序列。

通常将广义表表示为：

$$A = (a_1, a_2, \cdots, a_n)$$

其中，A 是名称，元素个数 n 是表的长度；若 a_i 不是原子，则称其为A的子表。

注意广义表的递归性。

如：

A = （a，（b，a，b），（），c，（（（2））））；

B = （）；

C = （e）；

D = （A，B，C）；

E = （a，E）；

重点：

1，

数据结构包括三方面内容：逻辑结构、存储结构和数据的运算。

逻辑结构分为线性结构和非线性结构。

存储结构，又称为映像，也称为物理结构。分为顺序存储、链式存储、索引存储、散列存储。

可以用抽象数据类型(ADT)定义一个完整的数据结构。

2，时间复杂度与空间复杂度的分析（）

3，有三种存储结构，亦即：顺序存储、链式存储和静态链表。

4，栈（递归）

5，队列，循环队列

6，串，KMP

7，数组，压缩存储：对称矩阵，对角矩阵（带状矩阵），稀疏矩阵